

Supplementary material for “It Is Not the Signature: An Empirical Study of a Hidden Key-Parse Cost in JSON Web Token Validation”

Jannatul Ferdous Katha, Tasmia Tahmid Prova, Md Kishor Morol, Tze Hui Liew, Dip Nandi

Abstract

This supplement contains material moved from the main paper for length: the extended related work, measurement-protocol details, additional analyses for RQ1 to RQ3, the full discussion of threats to validity, and a design note on relocating validation to the service mesh. Section, table and figure numbers without an “S” prefix refer to the main paper. Every value is generated from the replication package, as in the main paper.

S1. Extended related work

This section gives the full discussion of related work that Section 2 of the main paper summarises.

Where the cost is usually attributed. Published measurement of layered enforcement in microservices concentrates on transport and proxy layers. Zhu et al. (2023) decompose service-mesh sidecar overhead and show that the dominant contributors depend on configuration and workload rather than being a fixed tax; we reach an analogous conclusion one layer up, for application-level validation. Application-level token validation is itself thinly evidenced, and is commonly reported as a difference between scenario averages, which attributes to validation every difference between two runs. We were unable to locate a study that measures the key-resolution step directly; we state that as an absence we searched for rather than as a claim about the whole literature.

Cryptographic cost has been over-attributed before. Coarfa et al. (2006) remains the clearest precedent: measuring TLS web servers rather than primitives, they found the public-key operation was a minority of server cost. Work on

cryptographic library performance since has largely stayed at the level of the primitive or the protocol exchange (Naser et al., 2020; Sikeridis et al., 2020), with Tuveri and Brumley (2019) a notable exception in examining how a library loads and dispatches implementations rather than how fast those implementations run. Our result concerns neither: it is the cost of getting key material *into* the library.

Performance bugs as a class. Jin et al. (2012) studied real-world performance bugs across five code bases and found that most follow a small number of recurring patterns (among them uncoordinated and skippable function calls) and that fixes are usually local. Zaman et al. (2012) found, from 400 Firefox and Chrome reports, that performance bugs differ from other bugs in how they are reported, discussed and fixed, in part because reproducing them is hard; Nistor et al. (2013a) report that most are found by code reasoning rather than by observing symptoms. Liu et al. (2014) reach similar conclusions for mobile applications. Detection work targets structural symptoms: redundant traversals with similar memory-access patterns (Nistor et al., 2013b), or predicates that separate slow from fast runs (Song and Lu, 2014). The defect here is of the “skippable call” kind Jin et al. describe, but it has none of the structural signatures those detectors look for: it is one call, executed once per request, whose failure is its only observable effect. It is also the shape that makes a performance bug hard to report: our own upstream report was preceded by a two-year-old report of the symptom with no diagnosed cause (Section 8.6).

JavaScript and Node.js performance. Selakovic and Pradel (2016) classified performance issues in JavaScript projects and found that inefficient API usage (correct calls made in a costly way) is among the most common root causes, and that the fixes are typically a few lines. Their later profiler (Selakovic et al., 2017) targets precisely the *order* in which a program evaluates conditions, the axis on which this defect lives: a check that almost always fails is evaluated before one that almost always succeeds. Gong et al. (2015) locate code that defeats JIT optimisation; exception-heavy control flow is one such pattern, and Section 4.2 measures what it costs here. Work on the Node.js event loop shows why a per-request synchronous cost matters beyond its magnitude: a single slow handler delays every other client (Davis et al., 2018; Staicu and Pradel, 2018). We make no availability or capacity claim, but the mechanism by which a synchronous per-request cost propagates is the same.

Version and environment variation. That software performance changes across versions in ways maintainers do not anticipate is established: Kaltefleiter et al. (2023) track performance across configurable systems’ release histories, and Chen and Shang (2017) study the code changes that introduce regressions. Laaber and Leitner (2018) assess whether microbenchmark suites detect such changes at all, and Stefan et al. (2017) find performance unit testing rare in practice. Container images make this concrete by pinning a dependency set implicitly: Cito et al. (2017) characterise how the ecosystem uses them, Zerouali et al. (2019) show that images routinely ship outdated packages, and Malka et al. (2026) show that an image tag does not guarantee a reproducible build. For OpenSSL 3 specifically, the maintainers of a major downstream library have stated publicly that parsing and key loading regressed substantially against 1.1.1 (Gaynor and Kehrer, 2026); that statement is not peer reviewed and was not produced under a protocol we can assess, and Section 5.2 supplies a controlled measurement for the one path this paper concerns. Section 5 is an instance with a measured cost attached.

Measurement protocol is the thing at stake. The methodological core of this paper is not new. Georges et al. (2007) established that a single execution of a managed-runtime benchmark is not a measurement, and that intervals must be computed across executions rather than within one; Kalibera and Jones (2013) show how to spend a fixed budget across levels of repetition, which is the reasoning behind our choice of many short processes over few long ones. Mytkowicz et al. (2009) showed that environmental details unrelated to the code under study can produce effects larger than the one being measured, without any obvious mistake, and Curtsinger and Berger (2013) that memory layout alone can do so. Barrett et al. (2017) found that virtual machines frequently do not reach a steady state, Traini et al. (2023) that developers misjudge when warmup ends, and Costa et al. (2021) catalogue how microbenchmarks go wrong in practice. Mytkowicz et al. (2010) showed that sampling profilers disagree on which method is hot, which is why Section 4 uses the profiler only as corroboration of a result obtained by differencing and relies on ordering rather than on its percentages. Our contribution is not to these methods but is an instance of what they warn about. The attribution this paper overturns has exactly the structure Mytkowicz et al. describe: a second variable, the bundled OpenSSL version, moving unrecorded alongside the one under study, at a granularity coarse enough to hide it (Section 5).

Environment as a confounder. Laaber et al. (2019) quantify how far cloud environments degrade microbenchmark reliability, and Leitner and Cito (2016) characterise performance variation across public infrastructure providers. Their concern is variance between nominally identical environments. Ours is adjacent and, we think, less examined: a *systematic* difference between environments that are not identical and are routinely treated as though they were: two container base images differing in a bundled library version. Laaber et al. are also the reason the second-architecture repetition in Section 5.3 is reported as an ordering check rather than as a replication of magnitudes.

Cryptographic interfaces are hard to use correctly. The defect reported here is not a cryptographic weakness but a consequence of how a cryptographic interface is invoked, which is a well-documented class. Lazar et al. (2014) found that most of the cryptographic vulnerabilities they studied were misuses of libraries by applications rather than bugs in the libraries. Egele et al. (2013) found cryptographic misuse widespread in deployed applications and Georgiev et al. (2012) found certificate validation broken across a wide range of non-browser software; Nadi et al. (2016) attribute much of it to the interfaces themselves rather than to developer inattention, a position Green and Smith (2016) put directly and Indela et al. (2016) respond to with interface proposals; and Acar et al. (2017) show experimentally that API design materially changes whether developers produce correct code. Detection has been automated by specifying correct usage, as CrySL does (Krüger et al., 2018), and by static analysis at scale (Rahaman et al., 2019). Our finding sits within that line with one difference: the interface here admits a use that is *correct*, and costly only for a reason invisible at the call site. A CrySL-style rule could express “pass a key object, not a string”, but the rule sets we know of encode security properties only, so none would report it.

JWT and JOSE libraries. Security analyses of JWT implementations consistently find key-handling defects. Detering et al. (2017) demonstrated attacks, key confusion among them, against several popular JOSE libraries; Xu et al. (2023) automated the detection of cryptographic vulnerabilities in applications that use JWT; Yang et al. (2026) evaluated 43 JWT libraries across 16 programming languages for vulnerabilities; and Shatnawi et al. (2024) surveyed Python JWT libraries for security warnings and adoption. RFC 8725 (Sheffer et al., 2020) codifies the resulting practice. All of this work treats a library’s handling of key material as a security surface. Section 7

treats the same surface as a cost site, across seven libraries, and Section 8.4 shows the two views are coupled: the control flow that costs time in one library is the control flow that enforces its key-type check.

Mining public repositories. Section 6.3 draws on code hosted publicly, and the hazards of doing so are catalogued by Kalliamvakou et al. (2014), among them that many repositories are not software projects, that activity is not uniformly distributed, and that a search result is not a random sample. Our sampling frame is stated in Section 3.10 with those hazards in view, which is why no population proportion is computed from it.

The cost of exceptions. That exceptions are expensive in managed runtimes, and that their cost is concentrated in construction rather than in unwinding, is long established: Ogasawara et al. (2006) optimise around exception paths in a Java virtual machine, and Renwick et al. (2019) quantify the cost in a compiled setting. Parravicini and Mueller (2021) provide the closest available cost-attribution baseline for the V8 engine, though on speculation rather than exceptions. Empirical studies of how exceptions are *used* find handlers that discard what they catch to be common: Cabral and Marques (2007) report that handlers in the Java and .NET code they studied rarely attempt recovery, Kery et al. (2016) find that handlers mostly log, print, return or rethrow, and de Pádua and Shang (2017) measure the prevalence of handling anti-patterns, as Weimer and Necula (2004) did earlier for error-handling mistakes. That literature treats a swallowed exception as a correctness and maintainability smell. The handler at the centre of this paper is a swallowed exception used as control flow on a hot path, and it is a performance defect rather than a correctness one: the fallback it guards is correct. None of these studies measures such a handler’s cost; Section 4.2 does, and finds the exception itself the smaller part.

Key-type confusion. That a token declaring an HMAC algorithm must not be validated against asymmetric key material is long established; RFC 8725 (Sheffer et al., 2020) describes the attack and its mitigations. The library studied here has been hardened against this class twice. Versions before 4.2.2 accepted an HMAC token verified with an RSA or EC public key used as the shared secret (CVE-2015-9235) (GitHub Advisory Database, 2015), and versions up to 8.5.1 could still be led to verify forged tokens from RSA to HMAC through their key-retrieval path (CVE-2022-23541), alongside two related

defects in key-type and default-algorithm handling (CVE-2022-23539, CVE-2022-23540) (Auth0, 2022); all three were fixed in version 9.0.0. Section 8 reports no new attack. It reports that the second hardening introduced the control flow whose cost this paper measures, that the defence depends on that control flow, and that removing the cost naively removes the defence.

What we searched for and did not find

Three areas bear directly on this result and we were unable to locate verifiable peer-reviewed work in any of them. We state each as an absence we searched for rather than as a claim about the whole literature; each is also, in part, why the misattribution corrected here survived as long as it did. Two adjacent questions (the cost of error construction in V8, and where key-parsing cost lies inside OpenSSL) were equally unstudied, and this paper measures both directly (Sections 4.2 and 5.2).

OpenSSL 3.x performance regressions. We found no peer-reviewed study of performance regressions across the OpenSSL 3.x series. Evidence exists as a maintainer statement (Gaynor and Kehrer, 2026), as project issues (for instance `openssl/openssl` issues #17627 and #20698, which concern performance of the 3.x provider and decoder machinery) and as mailing-list discussion. None was produced under a measurement protocol we can assess, so Section 5.2 measures the path directly rather than adopting the mechanism those sources propose.

Base image to bundled library to application performance. We found no academic study tracing that chain. Malka et al. (2026) address build reproducibility, Zerouali et al. (2019) package staleness and Cito et al. (2017) ecosystem practice; none measures how the library version an image pins changes application performance.

Cryptographic API misuse as a performance defect. The misuse literature cited above treats misuse exclusively as a security problem. We found no work treating a correct-but-costly use of a cryptographic interface as a latency or throughput defect, which is precisely what this paper reports.

S2. Measurement protocol details

This section gives the details of the measurement protocol summarised in Section 3.2, the full descriptions of the C probe, the exception baseline and the in-situ instrument (Sections 3.5, 3.6 and 3.9), and the sampling frames of Section 3.10.

S2.1. Unit of measurement and interval estimation

An interval computed over the calls inside one process describes that process’s steady state and is far too narrow: it cannot observe the compilation decisions, inline-cache states and heap layouts that differ between runs of the same code (Georges et al., 2007). Conditions in the decomposition of Section 4 were run in 15 separate processes of 40000 calls; conditions in the runtime matrix of Section 5 in 10 processes of 20000 calls. The largest between-process coefficient of variation is 3.5% on the host; in the matrix it is 20.2%, occurring on the `probe_succeeds` condition under `node:26.6.0-bookworm` rather than across the matrix generally.

Bootstrap intervals over 15 or 10 units are coarse, and we report them as indicative rather than as calibrated coverage. Where an interval collapses to a point (the timer overhead row of Table 2), it should be read as the clock’s resolution showing through, not as a precise estimate. The per-process values underlying every interval are in the replication package.

Conditions are interleaved within each round rather than run in blocks. Under block ordering a thermal excursion or a background process lands entirely on whichever condition is executing and is indistinguishable from an effect; interleaved, it is distributed across conditions and appears as between-round variance. The clock’s own cost is measured in the same process as the condition it accompanies and is reported rather than subtracted: 0.042 μ s against the smallest quantity reported, 0.583 μ s.

Differences between conditions are reported throughout as the median of per-round *paired* differences, not as the difference of two medians. The two are not identical and the distinction matters at the magnitudes involved; Section 4.3 and Section 8.4 each rely on it.

S2.2. Taking the JavaScript engine out

Runtime pairs can separate engine from library only as far as stock builds allow, so the attribution was tested by removing the engine altogether. We read the Node source at the two release lines measured (`src/crypto/crypto_keys.cc` at v18.20.8; `deps/ncrypto` at v22.23.3, which issues the same sequence) and reproduced in C, call for call, what `createPublicKey()` asks of OpenSSL when handed an HMAC secret: three `PEM_bytes_read_bio` scans for a public key, an RSA public key and a certificate, each under an error-queue mark and each followed by `BIO_reset`; then, since none was recognised, `PEM_read_bio_PrivateKey` with Node’s passphrase callback; then the C side of Node’s error path, which drains

the OpenSSL error queue into strings. The V8 side of that error path is deliberately absent; it is what Section 3.6 measures.

The program was compiled once per OpenSSL release with one compiler (GCC 13.3.0) and one set of flags, against 8 releases from 3.0.16 to 3.6.1, each built from source with identical configuration and linked by run path so that no system library could be picked up. Each binary reports the version it actually loaded; rows whose loaded version differed from the one requested would be discarded, and there were 0. The protocol is that of Section 3.2: one condition per process, 15 processes per release and condition, 5000 timed calls each, timed with `clock_gettime`, conditions and releases interleaved within rounds. Besides the full sequence we time its parts separately (the three public-key scans, the private-key parse, the error-string drain) and a *successful* parse of an RSA-2048 public key through the same path.

S2.3. Separating the exception from the parse

Inside Node the failed parse is an OpenSSL failure followed by a V8 exception: Node constructs an **Error**, attaches **library**, **reason** and **code** properties, throws it through the `createPublicKey()` frame, and the caller discards it. To measure that exception without the parse, a JavaScript benchmark builds the same object (message, property values and error code are captured from a real failure on the runtime being measured, not typed) and throws it from 8 frames below the catch. Variants separate its parts: a plain **Error** thrown and caught in one frame; the Node-shaped error at depth; the same with stack capture disabled; and a Node-internal argument error raised before any OpenSSL call. The real probe is timed alongside, with and without stack capture, so that the stack's share of it is obtained by paired differencing. All conditions run under official Node release binaries, which bundle their own OpenSSL, on six runtimes (18.20.8, 20.20.2, 22.23.3, 23.11.1, 24.21.0 and 26.10.0) at 15 processes of 10000 calls per condition.

Combining this with Section 3.5 gives a three-way decomposition of the probe on each runtime: the C cost of the OpenSSL release that runtime bundles, the V8 exception, and a remainder we attribute to the binding between them: argument preparation, key-handle allocation and the C++ side of error construction. The three come from different processes and cannot be paired, so the interval on the remainder is obtained by resampling each run's processes independently.

S2.4. In-situ measurement

The containerised service (Section 3.1) exposes a histogram timing the verification call alone. The instrument is a *paired-scrape difference*: the cumulative histogram is read before and after each arrival-rate step, and the two readings are differenced, which isolates that step’s distribution from the cumulative counters. Means are computed as the difference of the histogram’s `_sum` divided by the difference of its `_count`, so they are exact and do not depend on bucket boundaries; bucket resolution bounds the reported percentiles only.

Seven arrival rates are driven, and the whole sweep is run twice on one process: once in ascending order and once in descending order. We report the *range across the two passes*. This is a range of two, not a confidence interval; no standard error is computed from it, and it is not written as a symmetric tolerance. The opposite-order design also serves as a drift test, which Section S5.1 reports.

One process and two passes is the whole of the in-situ evidence. It is sufficient to establish that the cost is present in the containerised service and of what order; it is not sufficient to characterise the shape of the cost-versus-rate relation, and Section 6.1 is written accordingly.

S2.5. The three prevalence frames

Population co-occurrence counts. Code-search counts over public repositories, in four cells: two languages by two import forms. The denominator of a cell is files importing the library; the numerator is files that *also mention* the pre-parsing interface. This measures co-occurrence of two strings in one file, not verified usage. The denominators are returned rounded (each is an exact multiple of 1024), so they are magnitudes rather than counts established file by file.

A classified sample of call sites. Call sites located by search, deduplicated to one file per repository so that no project dominates, with the key argument traced within its file. This is a convenience sample from a relevance-ranked, undocumented ranking; inclusion probabilities are unknown and no weighting back to the population is possible.

A targeted counter-search. Every file co-occurring the library with the pre-parsing interface, fetched in full and adjudicated by hand. This frame is deliberately biased toward finding the pattern (that is its purpose as a negative control) and it is therefore disqualified as a prevalence estimator.

S2.6. Host state is recorded per measurement

A service measured on a host it shares with its own load generator, ingress and sidecars is measured together with whatever else that host is doing. We observed this directly while building the instrument: re-running one configuration with a background indexing process consuming host CPU moved the point at which throughput collapsed by hundreds of requests per second, and both runs are committed in the replication package (Katha et al., 2026). A curve that moves with unrelated host activity does not characterise the service, and this paper therefore makes no capacity claim. Every in-situ window instead carries the host load average observed during it, committed beside the measurement, and Section S5.2 uses it.

S3. Additional analyses for RQ1

This section expands the checks on the decomposition summarised in Section 4.3.

S3.1. The parts do not account for all of it

If the penalty for supplying a string were exactly the discarded attempt plus the conversion, the two would sum to it. They do not, quite:

observed penalty	23.92 μ s
discarded parse + conversion	18.62 μ s
residual	5.292 μ s

All three quantities are medians of per-round paired differences, per Section 3.2. The residual’s interval, [4.916, 5.501], excludes zero. The two measured parts therefore account for 77.9% and not more; the remainder lies in the library’s own type and claim checking along the non-pre-parsed path. We report it rather than absorbing it into the headline, because a decomposition whose parts are declared to sum exactly to the whole has usually not been checked.

The pre-parsed path carries a smaller unexplained remainder of the same kind, and we record it for consistency rather than leaving the asymmetry unremarked: `verify()` with a pre-parsed key costs 6.042 μ s against 1.292 μ s of structural decode plus 2.166 μ s of HMAC, leaving roughly 2.58 μ s unattributed. Neither residual was measured directly; both are the subject of conditions we did not run, and both are stated as unattributed rather than assigned.

S3.2. Confirmation without differencing

The decomposition reaches its conclusion by subtracting conditions, the same inference style this paper criticises elsewhere. It is therefore checked against V8’s sampling profiler, which attributes self time to named frames with no arithmetic on our part. At 50000 iterations per run and 3 runs per environment, matched so the percentages are comparable, the asymmetric key parse accounts for a median 66.2% of self time on the host (range 65.3–67.4 across runs), 95.3% (95.1–95.6) under Node 18, and 61.5% (58.5–61.6) under Node 26. The frame does not appear among the top frames of the pre-parsed path in any environment.

Two methods, one answer. We rely on the ordering and the dominance rather than on the exact percentages: sampling attribution depends on rate and on what else the process is doing.

S4. Additional analyses for RQ2

This section expands Sections 5.1 and 5.3.

S4.1. Engine major version does not distinguish the two slow runtimes

Node 18 and Node 20 ship V8 10 and 11 (different major versions of the engine) and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μ s. Node 26 ships OpenSSL 3.5 and pays 22.94 μ s on the same libc. Within the OpenSSL 3.0 series, then, changing the engine’s major version does not move the quantity; had Node 20 been fast, the attribution would have been to the engine.

S4.2. What this design cannot separate

That argument establishes a negative within one OpenSSL series. It does not establish the positive attribution for the change between series, and we state the limitation rather than leaving it implicit.

The fast rows are Node 26. They carry OpenSSL 3.5 *and* V8 14 *and* every other change in that build. OpenSSL version is therefore perfectly confounded with engine version and with build configuration across the 3.0–3.5 boundary. Nothing in this matrix can separate them. The absence of variation between V8 10 and V8 11 does not license the inference that V8 14 is also irrelevant; that is a different comparison, and we did not make it.

What the matrix supports on its own is the following, and no more: the magnitude co-varies with the bundled OpenSSL version across four images;

the two slow images differ in engine major version and do not differ in magnitude; and the two libc variants of one image differ by less than the difference between series. Separating OpenSSL from the engine requires a crossed design: a runtime pairing a newer engine with an older OpenSSL, or the parse measured outside Node entirely. Section 5.2 reports both, and it is on them, not on this matrix, that the causal wording used in this paper rests.

S4.3. The effect is confined to key parsing

Across the four container rows, pre-parsed validation spans $1.26\times$, the HMAC primitive $1.39\times$ and structural decode $1.35\times$. The discarded parse spans $25.36\times$. The successful parse moves with it ($11.74\times$), so this is the cost of key parsing in general rather than anything peculiar to the failure path. The failure is simply the case in which an application pays that cost on every request for a result it discards.

S4.4. What this design does not isolate

A reader may notice that the uncontainerised host row is of the same order as the Node 26 container row. That comparison is confounded and we draw no conclusion about containerisation from it: the host runs a different operating system from the containers, so operating system, C library and Node build all change alongside the container boundary. Isolating containerisation would require a Linux-native baseline, which we did not obtain. The claims above concern only the four container rows, among which containerisation is constant.

S4.5. A second architecture

Table S1 repeats the runtime matrix, with the same images and scripts, on x86_64 (Section 3.8). Absolute values are not compared across the two machines; the question is whether the ordering holds.

The property the attribution rests on (every OpenSSL 3.0 image slower than every OpenSSL 3.5 image) holds on both architectures. The strict rank order does not: the two OpenSSL 3.0 images, Node 18 and Node 20, exchange places between ARM64 and x86_64. Their difference is small against the gap between series on either machine, and no claim in this paper depends on their relative order. On x86_64 the discarded parse spans $30.50\times$ across the images while pre-parsed validation spans $1.31\times$, against $25.36\times$ and $1.26\times$ on ARM64. The host decomposition of Table 2 was repeated too (Node 22.22.2, OpenSSL 3.5.5): the discarded parse is $19.42\ \mu\text{s}$ of a $36.48\ \mu\text{s}$ call (53%), the

Table S1: The runtime matrix of Table 4 on two architectures, same images and scripts, in μs . ARM64: 10 processes of 20000 calls. x86_64, a shared cloud virtual machine: 10 processes of 20000 calls. Absolute values are not comparable across the two machines.

Image	OpenSSL	Discarded parse		Pre-parsed <code>verify()</code>	
		ARM64	x86_64	ARM64	x86_64
<code>node:18.20.8-alpine</code>	3.0.16	373.79	607.1	7.58	12.66
<code>node:20-alpine</code>	3.0.19	420.62	556.3	7.25	11.89
<code>node:26.6.0-alpine</code>	3.5.7	22.94	42.13	6.56	10.94
<code>node:26.6.0-bookworm</code>	3.5.7	16.58	19.91	6.04	9.701

symmetric constructor 1.261 μs and the HMAC primitive 3.105 μs , so the component ordering of Table 2 holds there as well. The full x86_64 host table is in the replication package.

S5. Additional analyses for RQ3

This section tabulates the in-situ sweep plotted in Figure 6, analyses the difference between its two passes and the role of host load, and gives the full prevalence counts summarised in Section 6.3.

Table S2: In-situ validation cost with a string secret, by arrival rate. The two columns are the ascending and the descending pass over the same seven rates on one process; each is a single window, and the pair is two observations rather than an interval. Host load average during each window is committed with the measurement.

Arrival rate (requests/s)	Mean per call (μs)	
	ascending	descending
5	928.24	1027.97
10	976.28	1017.34
25	918.54	934.47
50	758.57	737.12
100	571.59	607.78
200	494.91	532.50
400	452.82	467.62

The decline with arrival rate is not interpreted. Cost falls monotonically from 928.24 μs at 5 requests/s to 452.82 μs at 400 requests/s. This is the opposite

of what contention alone would produce, and several mechanisms unrelated to validation would produce it: a fixed per-window overhead amortising over more calls, processor frequency scaling and idle-state exit latency at low rates, cache and branch-predictor residency, or continued tier-up in the engine. We did not run the conditions that would distinguish them. We therefore report the numbers and do not claim that validation cost depends on arrival rate; establishing that would require varying window duration at fixed rate, pinning the frequency governor, and extending warmup at low rates, none of which we did.

S5.1. The two passes differ systematically

Sweeping up and then down is not only replication; it detects warm-up, thermal and hysteresis effects. If the difference between passes were noise, its sign would alternate across rate points. It does not. In the string-secret sweep the descending pass is slower at 6 of 7 rate points; in the pre-parsed sweep the descending pass is *faster* at 7 of 7. A consistent sign across seven points is not what noise produces.

This has a direct consequence for how the ranges in Table S2 should be read. A systematic component means the two passes are not independent samples of a common value, so the range between them understates uncertainty further than a range of two already does. We therefore report ranges and draw no inference that depends on their width. It is also a methodological finding in its own right: a single-direction sweep would have produced a curve carrying a consistent bias with no way to detect it.

S5.2. Host load, not the condition, drives the outliers

Agreement between passes varies considerably across windows. We distinguish *relative* disagreement, the gap between passes as a fraction of their mean, from *absolute* disagreement, the gap in microseconds; the two behave differently and the distinction carries this subsection.

The widest relative disagreement in the dataset and the tightest occur at the same arrival rate in different sweeps. The wide one is the highest-load window in its own sweep on the descending pass, though not on the ascending pass, whose maximum load falls at a different rate. The tight one, at the same arrival rate in another sweep, sits at a lower load on both passes and its two passes agree to within $0.02\ \mu\text{s}$. Relative disagreement therefore does not track the condition under measurement.

We considered two alternative explanations for the relative asymmetry and reject both. A small-mean effect would require absolute disagreement to be roughly constant, so that its relative size grew only as means shrank; absolute disagreement instead varies over three orders of magnitude and is largest where the means are largest, which is the opposite pattern. A measurement floor would require the affected values to sit at the instrument’s resolution; they do not, and in any case the means reported here are computed from the histogram’s sum and count rather than from its buckets, so bucket resolution cannot bound them.

The operative consequence is that in-situ figures are reported with the host load recorded during their window, and comparisons between windows at different loads are not treated as comparisons between conditions.

Population co-occurrence. Of 790528 public JavaScript files importing the library through `require`, 29 also mention the pre-parsing interface; for JavaScript through `import` the figures are 524288 and 38; for TypeScript through `require`, 8288 and 0; and for TypeScript through `import`, 333824 and 101. Cells are not pooled into a rate: a file using both import forms would be counted twice, so their sum 1656928 is an upper bound on the union and the largest cell 790528 a lower bound. No cell exceeds 0.0303% of files mentioning the interface. These are search-engine estimates of file counts over public repositories only, they measure co-occurrence rather than use, and the denominators are returned *rounded* (each is an exact multiple of 1024), so they should be read as magnitudes rather than as counts established file by file.

Matching only the conventional `jwt.verify()` spelling would filter the corpus toward one import idiom, so call sites are found by resolving the local binding of the import, covering arbitrary aliases, namespace imports, destructured and renamed imports, and direct calls on the `require` expression. That binding resolution contributes 12 of the call sites above that the conventional spelling misses, of which 0 pass a pre-parsed key.

S6. Extended threats to validity

This section gives the full discussion that Section 10 summarises.

The limitations below are grouped by what they bear on: the measurements, the deployment evidence, the survey of public code, and the scope of the claims. Each states what the paper does not claim as a result.

S6.1. Measurement

Two architectures, one of them shared. The primary measurements come from a single ARM64 host. Section 5.3 repeats them on x86_64, and there the split between OpenSSL series and the component ordering of the host decomposition hold. But the x86_64 machine is a virtual machine on shared cloud infrastructure, whose absolute timings are known to be unreliable (Laaber et al., 2019); we therefore take only orderings and ratios between interleaved conditions from it, and no magnitude in this paper should be read as holding on x86_64 hardware in general. A repetition on a dedicated x86_64 host, which the replication package scripts in one command, would turn the ordering check into a replication of magnitudes. The C probe, the exception baseline and the cross-library comparison were measured only on the x86_64 machine. Their conclusions rest on ratios between conditions measured in the same rounds, which is the kind of quantity a shared machine can support, and the effects they report are far larger than the machine’s noise: the median between-process coefficient of variation is 1.5%, 2.9% and 2.8% in the three runs. The largest values (28.0%, 27.7% and 20.2%) occur on sub-microsecond or JIT-dominated conditions that carry no claim, and medians of per-process medians are insensitive to them. The C probe needs no JavaScript runtime and the scripts rebuild every OpenSSL release from source, so repeating it on the ARM64 host or a dedicated x86_64 machine is one command.

The C probe reproduces Node’s calls, not Node’s build. Section 5.2 issues the OpenSSL calls Node issues, as read from the Node source, against OpenSSL releases we built. Node builds its bundled OpenSSL with its own configuration, so the C figures approximate rather than reproduce the parse cost inside a given runtime, and the binding column of Table 3 inherits that approximation; it is slightly negative on two OpenSSL 3.0 runtimes for that reason. The causal claim does not inherit it: it compares releases under one build configuration with nothing else varying.

The exception baseline is a reconstruction. Node constructs the error from C++ through V8’s API; our baseline constructs an error of the same shape, message and properties from JavaScript at a comparable stack depth. The two paths share V8’s error construction and stack capture, and the paired stack-capture measurement on the real probe agrees in order of magnitude with the synthetic one, but they are not the same code path. The ARM64 host figure of Table 2 was not itself decomposed; Table 3 decomposes the same component on six other runtimes.

The headline host figure has no per-row OpenSSL version. The host decomposition of Section 4 predates the per-measurement OpenSSL field. The version in force is known from the run timestamp and a note recorded at the time: OpenSSL 3.6.3, which a package upgrade later that day moved to 3.6.4 under the same Node binary. We report it as inferred, marked as such in Table 4, rather than as recorded. Three measurements that do carry their version bound how much this matters. The same probe costs 18.01 μ s under Node 26.10.0 with OpenSSL 3.5.8, 19.42 μ s in the x86_64 host repetition with OpenSSL 3.5.5, and 16.58 μ s in the Node 26 container with OpenSSL 3.5.7: all of the same order as 18.04 μ s, and all from OpenSSL releases after the step of Table 5. The headline figure is therefore representative of a post-3.2 OpenSSL, which is all the paper uses it for; it would be wrong by more than an order of magnitude only if the host had run an OpenSSL 3.0 or 3.1 release, which the record excludes.

S6.2. Deployment measurement

Ranges of two, with a systematic component. The in-situ dispersion is the range across an ascending and a descending pass. It is not a confidence interval, and Section S5.1 shows the two passes differ systematically rather than randomly, so they are not independent samples of a common value and the range understates uncertainty further. No inference in this paper depends on the width of those ranges.

One in-situ process, two passes. The rate sweep rests on one process sweeping seven rates twice. That is sufficient for the claim made (the cost is present in the containerised service and of this order) and insufficient for any claim about the shape of the cost-versus-rate relation, which Section 6.1 accordingly does not make. The magnitude claim does not rest on the sweep alone: the A/B of Section 6.2 is a separate pair of windows, measured with a separate instrument, and it agrees with the microbenchmark prediction on processor time. The wall-clock excess over that prediction is attributed to time off the processor and not decomposed further; the decline with arrival rate has several candidate explanations unrelated to validation that we did not test (Section 6.1).

Host load, and the confound that motivated recording it. A throughput curve measured on a host shared with its own load generator moves with unrelated activity on that host, as Section S2.6 describes, which is why no capacity claim

appears in this paper. Every in-situ window here carries the load average observed during it, and Section S5.2 shows that window-to-window agreement tracks that load rather than the condition. Contention inflates a duration rather than deflating it, so in-situ figures are upper bounds; but comparisons between windows at different loads are not comparisons between conditions, and we do not treat them as such.

The A/B is two windows at unequal load. The two arms of Table 6 are consecutive windows rather than interleaved rounds, and their host load is known only at the instant each began. That bars comparing them on wall-clock time, and we do not. It does not bar the CPU comparison the section rests on: contention inflates elapsed time, whereas a container’s own cgroup accounting charges it only the processor time it used. Attaching the per-second host sampler to a re-run of both arms, interleaved, would remove the limitation; we did not.

S6.3. Survey of public code

Sampling frames are not interchangeable. The three prevalence instruments of Section 3.10 have different units and different selection mechanisms, and none supports a population proportion. The co-occurrence counts measure two strings appearing in one file, with a false-positive rate established at 5 of 13 on hand inspection. The classified sample has unknown inclusion probabilities, a limitation of repository search noted more generally by Kalliamvakou et al. (2014). The counter-search is selected on the outcome.

The matcher’s coverage. Call sites are found by resolving the local binding of the import, which covers aliases, namespace imports, destructured and renamed imports and direct calls on the require expression. It does not resolve bindings across files, so a project that constructs a pre-parsed key in one file and validates in another is invisible to this analysis. This is the strongest threat to Section 6.3; within-file tracing found no such case, which is weak evidence against it.

One rater. The 13 automatically flagged call sites were read and classified by the first author, and the authors are not disinterested: the paper’s claim is stronger the more of them are false positives, and 5 were classified that way. There was no second rater and no inter-rater agreement to report. The per-case verdicts, the evidence for each and the hashes of the files they were

read from are published so the adjudication can be repeated, which is weaker than having repeated it.

S6.4. Scope

Token caching is not in the comparison. A service that caches verified tokens pays this cost once per token rather than once per request. We did not measure such a configuration, so the per-request figures here do not describe a caching deployment.

The security analysis is from source. Section 8 establishes reachability and provenance by reading `verify.js` at the releases named and by running the library’s own tests; it does not drive forged tokens at the containerised service, time the patched library on tokens declaring a non-HMAC algorithm, or time version 8.5.1. Its statements are therefore about which inputs reach the attempt and when the code arrived, not about throughput under attack, and the paper makes no availability claim.

Seven libraries, one version each. Section 7 covers seven libraries in four languages, chosen as the most used in each ecosystem, each at one release. Other libraries, and other releases of these, may resolve keys differently; each classification is from source at the version measured. The comparison supports the claim that the cost follows a design choice rather than JWT validation; it does not estimate how many libraries make that choice.

S7. Relocation to the service mesh: a design note

S7.1. Relocation is not the first remedy

A cost believed to be cryptographic invites relocation to a sidecar or gateway. The cost measured here is not cryptographic and is largely avoidable in place, so an architectural remedy that also moves a trust boundary should not be the first thing reached for. We implemented that relocation for the service studied here; applying a mesh validation policy duplicates verification rather than moving it, and making it effective requires the application to trust a proxy-supplied claims header, a known anti-pattern that needs a filter stripping client-supplied copies. We report that only as a design note: it carries no measurement, and the configuration, the observation and the inconclusive automated probe are documented in the replication package (Katha et al., 2026).

References

- Acar, Y., Backes, M., Fahl, S., Garfinkel, S., Kim, D., Mazurek, M.L., Stransky, C., 2017. Comparing the usability of cryptographic APIs, in: 2017 IEEE Symposium on Security and Privacy (SP). doi:10.1109/SP.2017.52.
- Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update for jsonwebtoken. Auth0 security bulletin, 21 December 2022, <https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jwebtoken>. Accessed 2026-09-28.
- Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages* 1, 52:1–52:27. doi:10.1145/3133876.
- Cabral, B., Marques, P., 2007. Exception handling: A field study in Java and .NET, in: ECOOP 2007 – Object-Oriented Programming, Springer. pp. 151–175. doi:10.1007/978-3-540-73589-2_8.
- Chen, J., Shang, W., 2017. An exploratory study of performance regression introducing code changes, in: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). doi:10.1109/ICSME.2017.13.
- Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C., 2017. An empirical analysis of the Docker container ecosystem on GitHub, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR). doi:10.1109/MSR.2017.67.
- Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS web servers. *ACM Transactions on Computer Systems* 24. doi:10.1145/1124153.1124155.
- Costa, D., Bezemer, C.P., Leitner, P., Andrzejak, A., 2021. What’s wrong with my benchmark results? Studying bad practices in JMH benchmarks. *IEEE Transactions on Software Engineering* 47, 1452–1467. doi:10.1109/TSE.2019.2925345.
- Curtsinger, C., Berger, E.D., 2013. STABILIZER: Statistically sound performance evaluation, in: Proceedings of the 18th International Conference on

- Architectural Support for Programming Languages and Operating Systems (ASPLOS '13), pp. 219–228. doi:10.1145/2451116.2451141.
- Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript and Node.js: First-class timeouts as a cure for event handler poisoning, in: Proceedings of the 27th USENIX Security Symposium, pp. 343–359.
- Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017. On the (in-)security of JavaScript object signing and encryption, in: Proceedings of the 1st Reversing and Offensive-oriented Trends Symposium (ROOTS '17). doi:10.1145/3150376.3150379.
- Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical study of cryptographic misuse in Android applications, in: Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13). doi:10.1145/2508859.2516693.
- Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptography. pyca/cryptography project statement, <https://cryptography.io/en/latest/statements/state-of-openssl/>. Maintainer statement, not peer reviewed; accessed 2026-09-24.
- Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07). doi:10.1145/1297027.1297033.
- Georgiev, M., Iyengar, S., Jana, S., Anubhai, R., Boneh, D., Shmatikov, V., 2012. The most dangerous code in the world: Validating SSL certificates in non-browser software, in: Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12). doi:10.1145/2382196.2382204.
- GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-2015-9235, GHSA-c7hr-j4mj-j2w6). <https://github.com/advisories/GHSA-c7hr-j4mj-j2w6>. Accessed 2026-09-28.
- Gong, L., Pradel, M., Sen, K., 2015. JITProf: Pinpointing JIT-unfriendly JavaScript code, in: Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE '15). doi:10.1145/2786805.2786831.

- Green, M., Smith, M., 2016. Developers are not the enemy! the need for usable security APIs. *IEEE Security & Privacy* 14. doi:10.1109/MSP.2016.111.
- Indela, S., Kulkarni, M., Nayak, K., Dumitras, T., 2016. Helping Johnny encrypt: Toward semantic interfaces for cryptographic frameworks, in: *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2016)*. doi:10.1145/2986012.2986024.
- Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and detecting real-world performance bugs, in: *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12)*, pp. 77–88. doi:10.1145/2254064.2254075.
- Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in: *Proceedings of the 2013 International Symposium on Memory Management (ISMM '13)*. doi:10.1145/2464157.2464160.
- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D., 2014. The promises and perils of mining GitHub, in: *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR 2014)*. doi:10.1145/2597073.2597074.
- Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023. Performance evolution of configurable software systems: An empirical study. *Empirical Software Engineering* 28. doi:10.1007/s10664-023-10338-3.
- Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026. Replication package: a pre-authentication key parse dominates JWT validation in jsonwebtoken. doi:10.5281/zenodo.23011153. Source repository: <https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->.
- Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer practices for locally handling exceptions, in: *Proceedings of the 13th International Conference on Mining Software Repositories (MSR '16)*, pp. 484–487. doi:10.1145/2901739.2903497.
- Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An extensible approach to validating the correct usage of cryptographic APIs,

- in: 32nd European Conference on Object-Oriented Programming (ECOOP 2018), pp. 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.
- Laaber, C., Leitner, P., 2018. An evaluation of open-source software microbenchmark suites for continuous performance assessment, in: Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18). doi:10.1145/3196398.3196407.
- Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking in the cloud. how bad is it really? Empirical Software Engineering 24. doi:10.1007/s10664-019-09681-1.
- Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic software fail? A case study and open problems, in: Proceedings of the 5th Asia-Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.2637237.
- Leitner, P., Cito, J., 2016. Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds. ACM Transactions on Internet Technology 16. doi:10.1145/2885497.
- Liu, Y., Xu, C., Cheung, S.C., 2014. Characterizing and detecting performance bugs for smartphone applications, in: Proceedings of the 36th International Conference on Software Engineering (ICSE '14), pp. 1013–1024. doi:10.1145/2568225.2568229.
- Malka, J., Zacchiroli, S., Zimmermann, T., 2026. Docker does not guarantee reproducibility. arXiv preprint arXiv:2601.12811.
- Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09). doi:10.1145/1508244.1508275.
- Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2010. Evaluating the accuracy of Java profilers, in: Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '10), pp. 187–197. doi:10.1145/1806596.1806618.

- Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in: Proceedings of the 38th International Conference on Software Engineering (ICSE '16). doi:10.1145/2884781.2884790.
- Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang, Z., Yuan, X., 2020. Performance evaluation and modeling of cryptographic libraries for MPI communications. arXiv preprint arXiv:2010.06139.
- Nistor, A., Jiang, T., Tan, L., 2013a. Discovering, reporting, and fixing performance bugs, in: Proceedings of the 10th Working Conference on Mining Software Repositories (MSR '13). doi:10.1109/MSR.2013.6624035.
- Nistor, A., Song, L., Marinov, D., Lu, S., 2013b. Toddler: Detecting performance problems via similar memory-access patterns, in: Proceedings of the 35th International Conference on Software Engineering (ICSE '13), pp. 562–571. doi:10.1109/ICSE.2013.6606602.
- Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed optimization in Java. ACM Transactions on Programming Languages and Systems 28. doi:10.1145/1111596.1111598.
- de Pádua, G.B., Shang, W., 2017. Studying the prevalence of exception handling anti-patterns, in: Proceedings of the 25th International Conference on Program Comprehension (ICPC '17), pp. 328–331. doi:10.1109/ICPC.2017.1.
- Parravicini, A., Mueller, T., 2021. The cost of speculation: Revisiting overheads in the V8 JavaScript engine, in: 2021 IEEE International Symposium on Workload Characterization (IISWC). doi:10.1109/IISWC53511.2021.00013.
- Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantarcioglu, M., Yao, D.D., 2019. CryptoGuard: High precision detection of cryptographic vulnerabilities in massive-sized Java projects, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), pp. 2455–2472. doi:10.1145/3319535.3345659.
- Renwick, J., Spink, T., Franke, B., 2019. Low-cost deterministic C++ exceptions for embedded systems, in: Proceedings of the 28th International Conference on Compiler Construction (CC 2019). doi:10.1145/3302516.3307346.

- Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance profiler for optimizing the order of evaluations, in: Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '17). doi:10.1145/3092703.3092716.
- Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in JavaScript: An empirical study, in: Proceedings of the 38th International Conference on Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/2884781.2884829.
- Shatnawi, A., Al-Duwairi, B., Smarneh, A., 2024. Comprehensive empirical study of Python JWT libraries. *Procedia Computer Science* 238, 827–832. doi:10.1016/j.procs.2024.06.099.
- Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current Practices. Request for Comments RFC 8725. Internet Engineering Task Force. doi:10.17487/RFC8725.
- Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '20). doi:10.1145/3386367.3431305.
- Song, L., Lu, S., 2014. Statistical debugging for real-world performance problems, in: Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14), pp. 561–578. doi:10.1145/2660193.2660234.
- Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers, in: Proceedings of the 27th USENIX Security Symposium, pp. 361–376.
- Stefan, P., Horký, V., Bulej, L., Tůma, P., 2017. Unit testing performance in Java projects: Are we there yet?, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE '17), pp. 401–412. doi:10.1145/3030207.3030226.
- Traini, L., Cortellessa, V., Di Pompeo, D., Tucci, M., 2023. Towards effective assessment of steady state performance in Java software: Are we there yet? *Empirical Software Engineering* 28. doi:10.1007/s10664-022-10247-x.

- Tuveri, N., Brumley, B.B., 2019. Start your ENGINes: Dynamically loadable contemporary crypto, in: 2019 IEEE Cybersecurity Development (SecDev). doi:10.1109/SecDev.2019.00014.
- Weimer, W., Necula, G.C., 2004. Finding and preventing run-time error handling mistakes, in: Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '04). doi:10.1145/1028976.1029011.
- Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L., 2023. JWTKey: Automatic cryptographic vulnerability detection in JWT applications, in: Computer Security – ESORICS 2023, Springer. doi:10.1007/978-3-031-51479-1_14.
- Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., Wang, B., 2026. Token time bomb: Evaluating JWT implementations for vulnerability discovery, in: Proceedings of the Network and Distributed System Security Symposium (NDSS '26).
- Zaman, S., Adams, B., Hassan, A.E., 2012. A qualitative study on performance bugs, in: Proceedings of the 9th IEEE Working Conference on Mining Software Repositories (MSR '12), pp. 199–208. doi:10.1109/MSR.2012.6224281.
- Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On the relation between outdated Docker containers, severity vulnerabilities, and bugs, in: Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '19), pp. 491–501. doi:10.1109/SANER.2019.8668013.
- Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He, P., Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023. Dissecting overheads of service mesh sidecars, in: Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23). doi:10.1145/3620678.3624652.